UNION in SQL

A **UNION** in SQL is an operator used to **combine the results of two or more SELECT statements into a single result set**.

Unlike JOINs, which combine tables side-by-side using columns, UNION combines query results **vertically (row-by-row)**.

Key Characteristics of UNION

- Combines rows from multiple queries
- Each SELECT must return:
  - the same number of columns
  - compatible data types
- Column names come from the first SELECT statement
- UNION removes duplicates by default
- UNION ALL keeps duplicates

Short Notes on Each Script

1. Database Selection

```
USE techsphere_solutions;
```

What it does:

- Activates the TechSphere database
- Ensures all queries run in the correct schema

2. Basic UNION (Learning Demonstration)

```
SELECT first_name, last_name
FROM employees

UNION

SELECT role_title, base_salary
FROM job_roles;
```

What it does:

- Combines employee names with job role data
- Results are stacked vertically

Important Note:

- This is only for demonstration
- Mixing unrelated data is not recommended in real systems

3. UNION DISTINCT (Default UNION Behavior)

```
SELECT first_name, last_name
FROM employees

UNION

SELECT role_title, base_salary
FROM employees;
```

What it does:

- Combines results from identical queries
- Removes duplicate rows automatically

Key Concept:

UNION = UNION DISTINCT

4. UNION ALL

```
SELECT first_name, last_name
FROM employees

UNION ALL

SELECT role_title, base_salary
FROM employees;
```

What it does:

- Combines datasets
- Keeps all duplicate rows

Benefit:

- Faster than UNION
- Useful when duplicates are needed

5. Practical Business Use Case

Senior Employee Classification

```
WHERE TIMESTAMPDIFF(YEAR, date_of_birth, CURDATE()) > 50
```

What it does:

- Identifies employees older than 50
- Labels them as:

Senior Employee

Experienced Employee Classification

```
WHERE TIMESTAMPDIFF(YEAR, date_of_birth, CURDATE()) BETWEEN 40
AND 50
```

What it does:

- Identifies mid-aged experienced employees
- Labels them accordingly

High Salary Employee Classification

```
WHERE jr.base_salary >= 100000
```

What it does:

- Finds employees earning high salaries
- Uses JOINs to connect:
    - Employees
    - Positions
    - job roles

6. ORDER BY with UNION

```
ORDER BY first_name;
```

What it does:

- Sorts the final combined UNION result alphabetically

7. UNION Rules Section

This section documents important SQL UNION rules.

Rule 1

All SELECT statements must return:

- same number of columns

Rule 2

Data types must be compatible.

Rule 3

Column names come from first SELECT.

Rule 4

- UNION removes duplicates
- UNION ALL keeps duplicates

8. Business Insight Section

This explains real-world uses of UNION such as:

- HR analytics
- Employee classification
- Reporting dashboards
- KPI segmentation
- Data consolidation

Overall, Purpose of the Script

This script demonstrates how SQL UNION operations are used to:

- Combine datasets vertically
- Merge analytical results
- Create categorized reports
- Consolidate data from multiple queries
- Build business intelligence outputs

Key Learning Outcomes

After this script, you understand:

- Difference between UNION and JOIN
- Difference between UNION and UNION ALL
- How to merge multiple SELECT queries
- How to classify business data using UNION
- How SQL handles duplicates

Summary Insight

UNION is used to combine query results vertically into one dataset, making it useful for reporting, categorization, and consolidating related analytical outputs from multiple SQL queries.